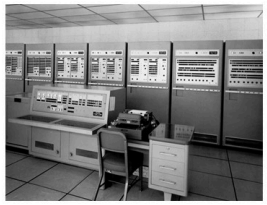
第12章 分布式系统安全

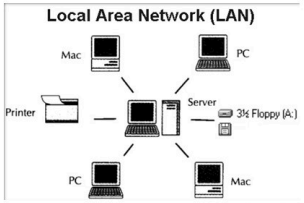
PPT版权：徐恪（清华计算机系）、李琦（清华网研院）

分布式系统的起源

- 并行计算：Burroughs Corporation在1962年推出了D825，这是一种四处理器计算机，可以访问多达16个共享的内存模块
- ARPAPNET：互联网的前身之一，于20世纪60年代末被推出，分布式系统开始拓展到主机这一粒度
- LAN：1970年，以太网等局域网技术的出现，标识着分布式系统开始广泛部署







[通信（点到点）→ 网络（目的：端到端数据传输）] → 分布式（合作计算）

分布式系统飞速发展

- 随着互联网技术的不断演进与用户数量的“爆炸式”增长，导致用户需求不断增加
- 基于单一节点的C/S架构所构建的服务无法承受如今的用户规模，分布式系统规模也在不断扩张

性能



日点击量峰值超1500亿次 12306是如何应对下来的?

2018-02-08 22:51

一年一度的春运如期而至，中国又将迎来“人类最大规模的周期性迁徙”。每年的春运，对售票系统12306也是一场艰难的大考。

2018年在线流媒体的订阅用户人数达到6.13亿，全球媒体巨头纷纷入局视频流媒体[图]

2019年09月23日 14:19:25 字号: T | T

根据数据，在线流媒体2018年的订阅用户数达到6.13亿，同比增长27%（增量1.31亿），成功超越有线电视5.56亿订阅用户，用户数4年翻4倍。

分布式系统无处不在

如今的我们每天都在和分布式系统打交道，例如：浏览网页用到的DNS系统，微信、搜索引擎以及淘宝、京东等购物平台，比特币、以太坊等区块链平台



容错/安全

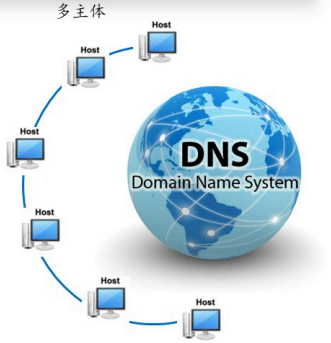








多主体



雨课堂 Rain Classroom

分布式系统是什么？

这是表象不是本质，本质是无法命运共享

如今所说的分布式系统通常是由分布在不同地理位置的系统组件所构成，这些系统组件通过网络传递消息完成动作协调，从而相互协作完成共同的任务

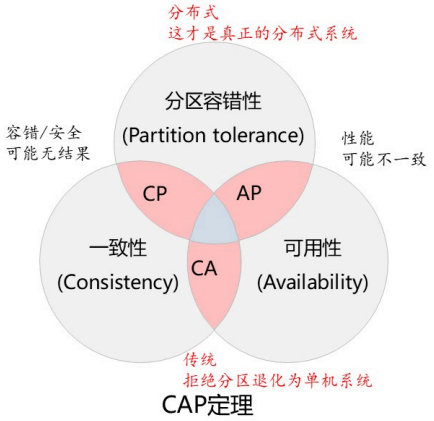
- 分布式系统的组成：
  - 交互网络 — 实现协作的前提，系统组件基于交互网络实现消息交互
  - 分布式算法 —协调分布式组建之间的交互，确保系统稳定运行
  - 信任机制 — 解决不同系统组件之间的信任问题，实现可信协作







分布式系统的三个属性



CAP定理

- **一致性**：分布式系统中各正确节点的所保存的同一变量状态信息都是一致的
- **可用性**：分布式系统在收到客户端的请求后，必须给出回应，不能让客户端陷入无限等待过程中
- **分区容错性**：分布式系统容忍其中节点出现分区，即一个区域中节点发往另一个区域中节点的数据包全部丢失，区域间无法进行通信

分布式系统安全问题的根源

安全与容错不一样

安全问题可能会造成系统分区，或是危害系统的一致性、有效性和隐私性

- 从分布式系统的组成来看，其安全问题的根源体现在交互网络、分布式算法和信任三个方面：
  - 攻击者可以攻击交互网络，监听交互过程中的隐私信息或造成网络分区
  - 协作过程中可能会出现冲突，故障等问题，从而影响一致性和可用性
  - 系统内部可能存在恶意组件，系统和客户端交互时也存在信任问题







主要内容

	安全问题根源	安全隐患	解决思路	具体措施
网络安全	**交互网络**	①消息重复与丢失 ②消息泄漏 ③消息篡改和伪造 ④消息重放 ⑤通信故障 ⑥日蚀攻击	建立安全、稳定的交互网络，降低分区出现的概率，确保系统组件之间能够安全、稳定地交互	①建立TCP连接 ②协商会话密钥 ③建立TLS连接 ④时间戳、挑战等机制 ⑤建立应用层路由 ⑥安全、稳定的邻居选择机制
分布式	**分布式算法**	①时钟问题 ②并发问题 ③节点故障	解决协作过程中的冲突与故障，确保协作过程中的稳定	①时钟同步算法 ②排序、锁等机制 ③故障容错的一致性算法
	信任	①非法访问 ②共享恶意资源 ③恶意组件	建立信任关系，解决系统对客户端、客户端对系统以及系统组件之间的信任问题	①零信任模型与访问控制 ②信用评价模型 ③拜占庭容错的一致性算法

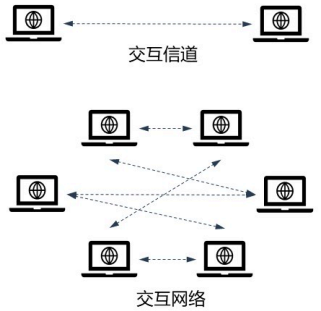
第1节 交互网络安全

- **交互网络的组成**
- 交互信道安全
- 弹性覆盖网络
- 防御日蚀攻击

交互网络的组成

在分布式系统中，节点之间可以通过交互信道实现消息传递；分布式节点选择邻居节点并建立交互信道，从而构成一个交互网络

- 交互信道：两个主机通过交互信道实现host-to-host的通信，常见的交互信道有UDP、TCP和TLS等
- 邻居选择机制：每个节点依靠邻居选择机制选取系统中的部分或全部节点建立交互信道，从而建立邻居关系，节点可通过邻居节点与其它非邻居节点交互



第1节 交互网络安全

- 交互网络的组成
- **交互信道安全**
- 弹性覆盖网络
- 防御日蚀攻击

第1节 交互网络安全

- 交互网络的组成
- 交互信道安全
- **弹性覆盖网络**
- 防御日蚀攻击

雨课堂 Rain Classroom

Resilient Overlay Networks

David Andersen, Hari Balakrishnan, Frans Kaashoek, and Robert Morris
MIT Laboratory for Computer Science
ron@nms.lcs.mit.edu
http://nms.lcs.mit.edu/ron/

18th ACM Symp. on Operating Systems Principles (SOSP)
October 2001, Banff, Canada.

Abstract

A Resilient Overlay Network (RON) is an architecture that allows distributed Internet applications to detect and recover from path outages and periods of degraded performance within several seconds, improving over today's wide-area routing protocols that take at least several minutes to recover. A RON is an application-layer overlay on top of the existing Internet routing substrate. The RON nodes monitor the functioning and quality of the Internet paths among themselves, and use this information to decide whether to route packets directly over the Internet or by way of other RON nodes, optimizing application-specific routing metrics.

Results from two sets of measurements of a working RON deployed at sites scattered across the Internet demonstrate the benefits of our architecture. For instance, over a 64-hour sampling period in March 2001 across a twelve-node RON, there were 32 significant outages, each lasting over thirty minutes, over the 132 measured paths. RON's routing mechanism was able to detect, recover, and route around *all* of them, in less than twenty seconds on average, showing that its methods for fault detection and recovery work well at discovering alternate paths in the Internet. Furthermore, RON was able to improve the loss rate, latency, or throughput perceived by data transfers; for example, about 5% of the transfers doubled their TCP throughput and 5% of our transfers saw their loss probability reduced by 0.05. We found that forwarding packets via at most one intermediate RON node is sufficient to overcome faults and improve performance in most cases. These improvements, particularly in the area of fault detection and recovery, demonstrate the benefits of moving some of the control over routing into the hands of end-systems.

and its constituent networks, usually operated by some network service provider. The information shared with other providers and AS's is heavily filtered and summarized using the Border Gateway Protocol (BGP-4) running at the border routers between AS's [21], which allows the Internet to scale to millions of networks.

This wide-area routing scalability comes at the cost of reduced fault-tolerance of end-to-end communication between Internet hosts. This cost arises because BGP hides many topological details in the interests of scalability and policy enforcement, has little information about traffic conditions, and damps routing updates when potential problems arise to prevent large-scale oscillations. As a result, BGP's fault recovery mechanisms sometimes take many minutes before routes converge to a consistent form [12], and there are times when path outages even lead to significant disruptions in communication lasting tens of minutes or more [3, 18, 19]. The result is that today's Internet is vulnerable to router and link faults, configuration errors, and malice—hardly a week goes by without some serious problem affecting the connectivity provided by one or more Internet Service Providers (ISPs) [15].

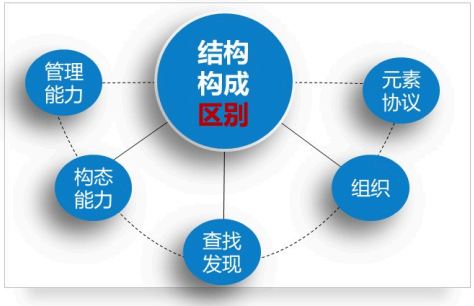
Resilient Overlay Networks (RONs) are a remedy for some of these problems. Distributed applications layer a "resilient overlay network" over the underlying Internet routing substrate. The nodes comprising a RON reside in a variety of routing domains, and cooperate with each other to forward data on behalf of any pair of communicating nodes in the RON. Because AS's are independently administrated and configured, and routing domains rarely share interior links, they generally fail independently of each other. As a result, if the underlying topology has physical path redundancy, RON can often find paths between its nodes, even when wide-area routing Internet protocols like BGP-4 cannot.

第1节 交互网络安全

- 交互网络的组成
- 交互信道安全
- 弹性覆盖网络
- **防御日蚀攻击**

问题背景：对等网络技术P2P

P2P (Peer-to-Peer)又称对等网络技术，也是建立在Internet之上的分布式Overlay网络，能够不依赖中心节点而只依靠对等协作实现资源发现与共享



相同

- 都能运行在不同（Internet / Intranet）平台
- 都能服务传统或新的应用：eBusiness eServices ...

思考：
P2P网络与分组交换网 有什么 根本上 差别？

①事物本身　②被认可的技术方案

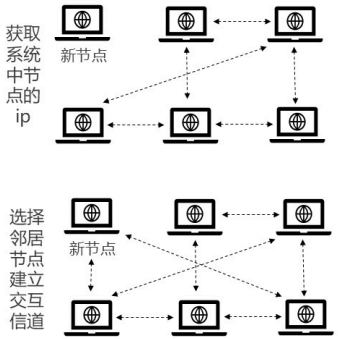
目标 约束 方案 分层

雨课堂 Rain Classroom

问题背景： P2P网络的建立流程

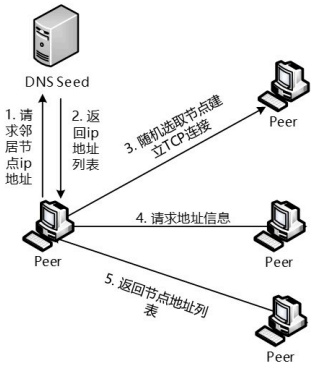
P2P网络的建立主要包含网络节点发现和邻居节点维护两部分

- 网络节点发现
  - 新加入的节点获取系统中其它节点的ip地址，并存储在本地的网络信息表中
- 邻居节点维护
  - 节点从本地的网络信息表中选择ip地址，并请求建立邻居关系



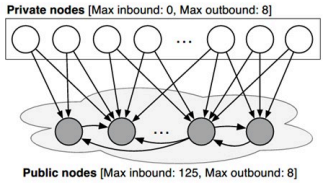
问题背景：网络节点发现

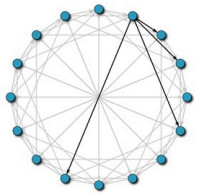
- 程序硬编码
  - P2P客户端程序中写入权威节点ip，用于客户端建立初始连接
- DNS seed
  - 向Seed服务器请求节点ip地址
- 向邻居请求
  - 已存在邻居后，和邻居节点定期交换所发现的新ip地址
- 节点查询
  - 针对特定的节点id，查询其ip地址，主要在有结构化的P2P网络中存在



问题背景：邻居节点维护

- 无结构化
  - 从发现的ip地址中，随机选择邻居
  - 通常限制节点的入向和出向连接数，入向指被请求建立的邻居关系，出向指主动请求建立的邻居关系
- 有结构化
  - 每个节点都有指定的id，并存在相应的公私钥用于id的认证
  - 节点基于id之间的距离按规定确定邻居节点
- 混合模式：有结构化P2P网络中，允许节点随机选择部分邻居节点

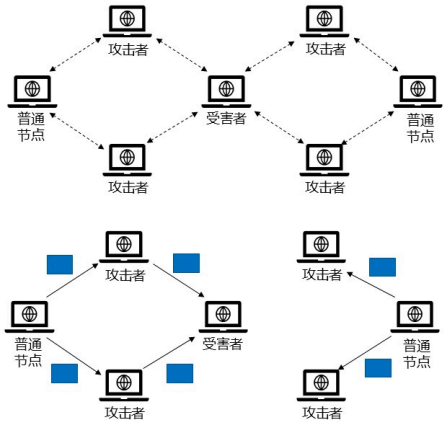




P2P安全威胁：日蚀攻击

攻击目标：针对网络节点发现以及邻居节点维护两个过程进行攻击，使得受害节点的所有邻居节点都是由攻击者所控制的恶意节点。

危害：攻击者可以选择性的转发对攻击者有利的消息给受害者，从而配合其它网络攻击；攻击者可以完全隔离受害者与网络中其它节点的信息交互



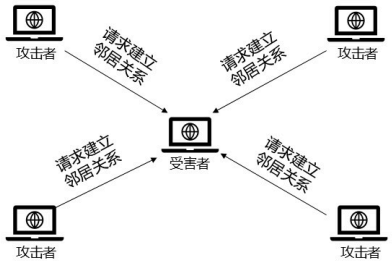
日蚀攻击流程：挤占入向连接

主要思路

- 攻击者控制大量节点主动向受害者请求建立邻居关系
- 若受害者已经和其它节点建立邻居关系，则可配合DDoS等其它网络攻击触发受害节点的重启，然后迅速挤占其入向连接

解决思路：

- 周期性更换邻居节点



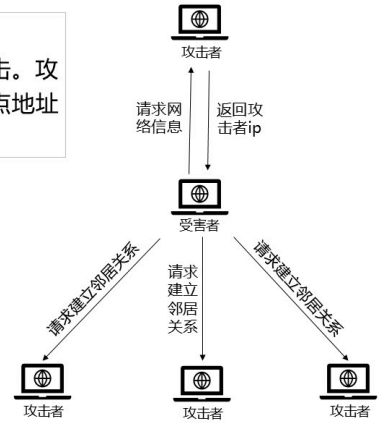
日蚀攻击流程：挤占出向连接

主要思路：

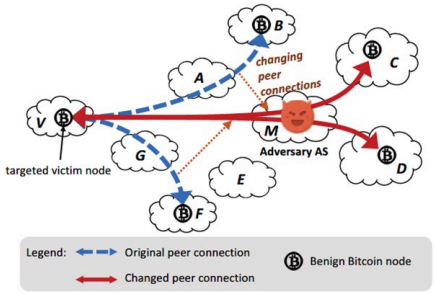
针对受害节点发现网络节点的过程进行攻击。攻击者向受害者发送大量由自己所控制的节点地址信息，等待受害者主动请求建立连接

解决思路：

- 增加网络信息表项的大小
- 增加出向连接个数
- 选择少量不易受攻击的可信权威节点作为邻居节点



日蚀攻击实例：基于路由劫持的攻击



主要思路：

- 向受害节点本地存储的节点地址列表注入大量影子ip（与受害节点间的通讯链路经过攻击者所控制的AS）
- 使受害节点与影子ip建立邻居关系，从而劫持受害者的所有邻居链路

解决思路：

- 邻居选择时将AS拓扑考虑进去

注. 图片来自来源:
Tran M, Choi I, Moon G J, et al. A stealthier partitioning attack against bitcoin peer-to-peer network[C]//2020 IEEE Symposium on Security and Privacy (SP). IEEE, 2020: 894-909.

第2节 分布式算法

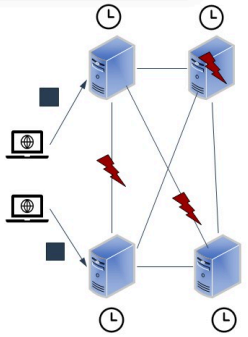
- **协作过程中的安全隐患**
- 时钟同步问题
- 并发问题
- 故障容错的问题

雨课堂 Rain Classroom

协作过程中的安全问题

分布式算法的目的是协调分布式组件之间的交互和动作，从而确保它们能够稳定协作实现一个共同的任务

- 缺乏全局时钟：各节点本地时钟可能不同步
  - 物理时钟：基于时钟同步机制完成物理时钟同步
  - 逻辑时钟：基于共识对事件发生的顺序达成一致
- 并发：同一时刻，系统中的不同组件可能面临来自多个客户端的大量并发请求
- 故障：系统组件在协作过程中可能出现故障，从而导致信息的短暂丢失

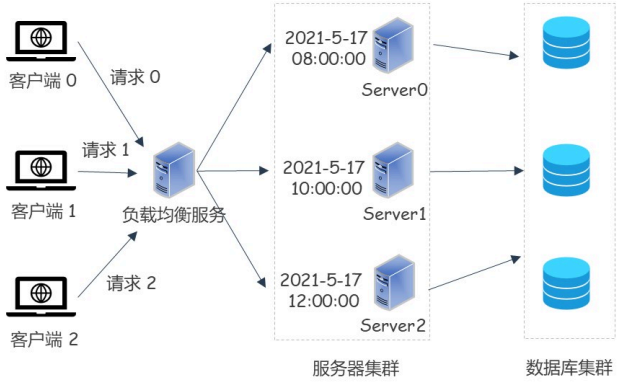


第2节 分布式算法

- 协作过程中的安全隐患
- **时钟同步问题**
- 并发问题
- 故障容错的问题

时钟不同步可能引发的问题

- 三个客户端几乎同时发送请求
- 请求由时钟不同步的服务器处理
- 服务器生成日志，时钟不同步导致三个请求所标记的时间相差大



时钟同步算法

- 主要思想：若节点A想与节点B完成时钟同步，则请求节点B发送本地时间T；然后预测A与B的网络时延RTT；之后将时间设置为T+RTT/2；RTT预测的越精确，时钟同步的误差越小



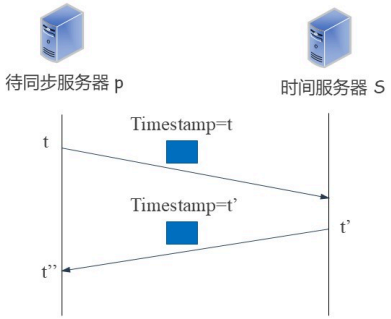
- 主要方式
  - Cristian算法：使用一个时间服务器，其它节点基于该服务器完成时钟同步，主要用于企业网内部
  - Berkeley算法：主要用于分布式集群系统中各节点之间时钟同步，主要用于企业网内部
  - NTP协议：主要运用于互联网，确保网络节点之间能够时钟同步

雨课堂 Rain Classroom

Cristian算法

基于一个接收UTC（通用协调时间）信号的外部时间服务器完成时钟同步

- 当服务器 P收到数据包时，本地时钟为t''，可得消息来回延迟为：
  $$T_{round} = t'' - t$$
- 在服务器 P 看来，当S收到消息时，P本地预计时间为 $t + T_{round}/2$，所以 P 和 S 的时间差为：
  $$t_s - t_p = t' - (t + T_{round}/2)$$
- 所以 P 在收到消息后将时钟设置为：
  $$t' + (t'' - t)/2$$

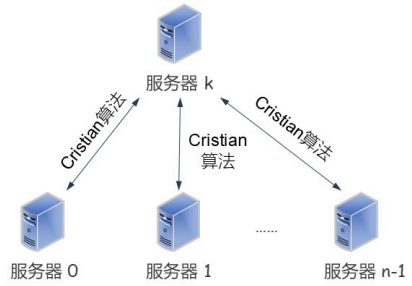


Berkeley算法

主要用于分布式集群服务器内部完成时钟同步

Berkeley算法的具体流程：

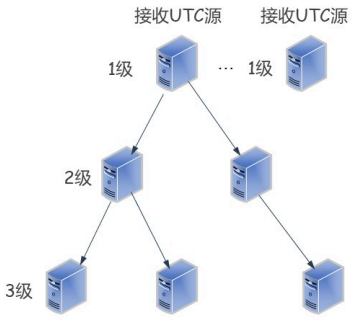
- 所有服务器与其它服务器间分别运行一次Cristian算法，根据本地的时间可得到n个不同的时间：
  $$t_0, t_1, \ldots, t_{n-1}$$
- 取平均数，获得本地最终时间：
  $$t = \sum_i t_i / n$$



网络时间协议（NTP）

NTP协议定义了时间服务的体系结构，以及如何在互联网上发布时间信息

- 主要目标：
  - 互联网的用户能够精确地与UTC同步
  - 提供稳定的时间服务，部分服务器出现故障时，用户依然能够完成时钟同步
  - 避免恶意攻击者干扰时钟同步过程
- 整体架构
  - 采用分层架构，1级服务器接收UTC源
  - 基于认证确保时间信息来自权威服务器
  - 同步源故障后，选择其它源作为替代



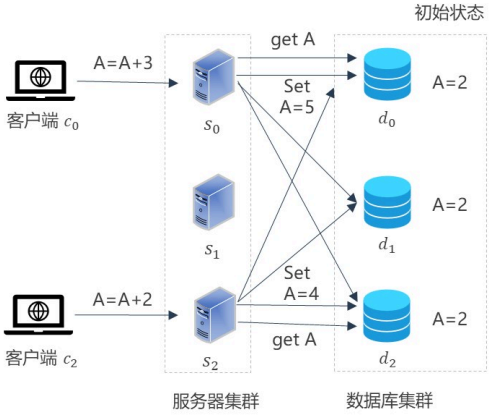
第2节 分布式算法

- 协作过程中的安全隐患
- 时钟同步问题
- **并发问题**
- 故障容错问题

雨课堂 Rain Classroom

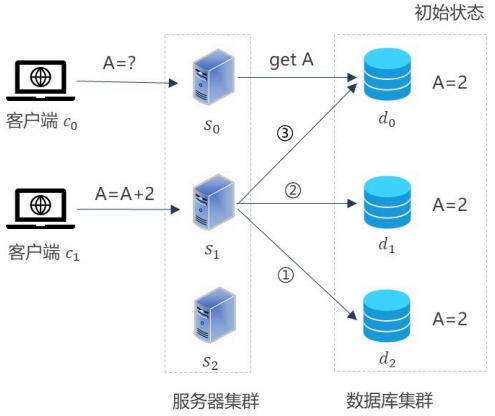
并发问题举例：破坏一致性

- 假设：状态A在三个数据库冗余存储，不存在并发控制措施
- 执行步骤
  - c_0向s_0请求将A的值加3
  - c_2向s_2请求将A的值加2
  - s_0和s_2分别向d_0和d_2读取A的值并完成计算
  - s_0和s_2依次向三个数据库更新A的值
- 由于指令的并发执行， 可能导致三个数据库A的值不一致



并发问题举例：指令之间相互影响

- 假设：状态A在三个数据库冗余存储，不存在并发控制措施
- 执行步骤
  - c_0向s_0请求A的值
  - c_1向s_1请求将A的值加2
  - s_1依次将三个数据库中A的值修改为4
  - s_0向数据库d_0读取A的值
- 由于指令的并发执行， s_0读取的值可能为2，也可能为4



事务与分布式事务

事务是原子的一组请求，这组请求不受其它客户端干扰，并且要么全部完成，要么不对服务器造成任何影响

- 事务需要满足ACID特性
  - 原子性（Atomicity）：操作必须全部完成，或者不留下任何结果
  - 一致性（Consistency）：将系统从一个一致状态转换到另一个一致状态
  - 隔离型（Isolation）：每个事务的执行不受其它事务影响
  - 持久性（Durability）：一旦事务完成，其效果将被保存到持久存储中

分布式事务是访问由多个服务器管理的对象的事务

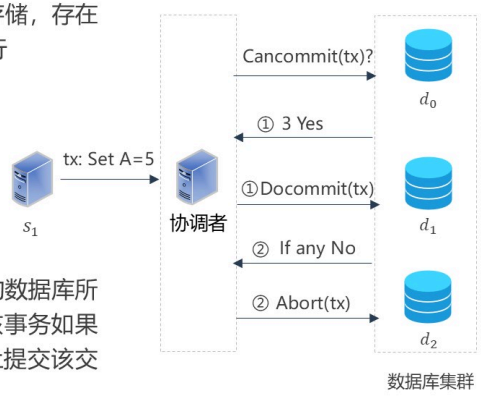
原子性 锁（通用）、提交，版本历史、日志与回滚、检查点
隔离 锁、序列化，两阶段锁
隔离：两阶段提交（阻塞、两将军问题）
持久性（写读连贯性）：容错（数据完整性、复制、多副本的问题）
一致性（与具体的数据库约束有关）

雨课堂 Rain Classroom

解决方案：两阶段提交确保原子性

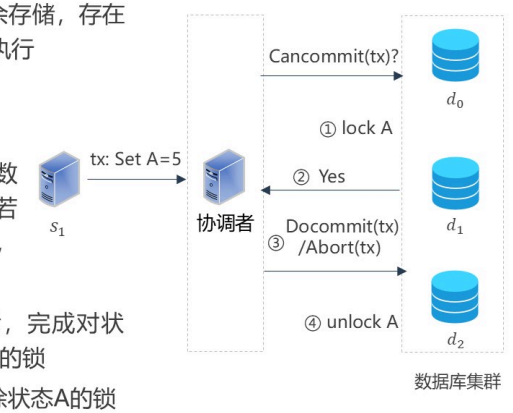
阻塞式协议

- 假设：状态A在三个数据库冗余存储，存在一个协调者引导二阶段提交的执行
- 执行步骤
  - s_1向协调者请求将A设为5
  - 协调者向所有存储状态A的数据库询问是否可提交该事务；若该设置不冲突，则返回Yes
  - 当协调者收到所有存储状态A的数据库所发送的Yes后，通知它们提交该事务如果收到任何No，则通知它们停止提交该交易



解决方案：基于锁机制确保指令执行隔离

- 假设：状态A在三个数据库冗余存储，存在一个协调者引导二阶段提交的执行
- 执行步骤
  - s_1向协调者请求将A设为5
  - 协调者向所有存储状态A的数据库询问是否可提交该事务若该设置不冲突，则返回Yes，并对状态A上锁
  - 若数据库收到Docommit后，完成对状态A的设置，然后解锁状态A的锁
  - 若数据库收到Abort后，解除状态A的锁



讨论

在两阶段提交过程中出现故障怎么办?

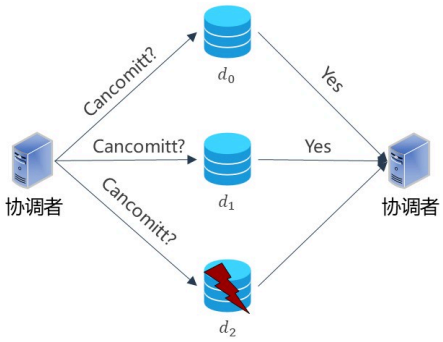
第2节 分布式算法

- 协作过程中的安全隐患
- 时钟同步问题
- 并发问题
- **故障容错问题**

雨课堂 Rain Classroom

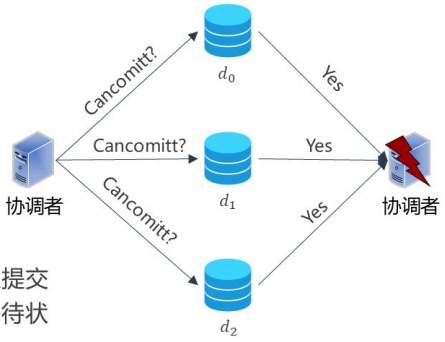
数据库节点故障举例

- 存在一个协调者和三个存储相同状态变量A的数据库，其中一个数据库出现故障
- 执行步骤
  - 协调者向三个数据库询问是否可以提交一个更改A值的交易
  - 数据库d_0和d_1向协调者返回Yes
  - 数据库d_2出现故障，没有反应
  - 协调者无法收到足够的Yes，陷入无限等待，或者采用计时机制，超时后通知所有节点停止提交



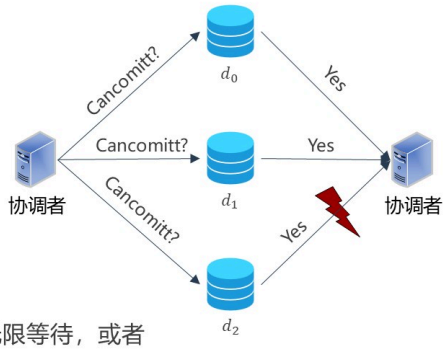
协调者故障举例

- 存在一个协调者和三个存储相同状态变量A的数据库
- 执行步骤
  - 协调者向三个数据库询问是否可以提交一个更改A值的交易
  - 三个数据库都向协调者返回Yes
  - 此时协调者出现故障
  - 数据库一直无法等到协调者的确认提交或停止提交消息，因此一直处于等待状态，且相关资源一直处于上锁状态



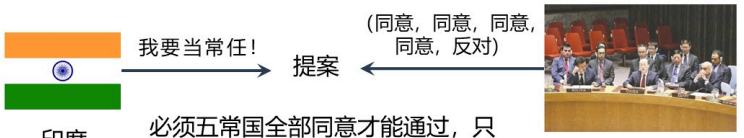
网络拥塞及链路故障导致的故障

- 存在一个协调者和三个存储相同状态变量A的数据库
- 执行步骤
  - 协调者向三个数据库询问是否可以提交一个更改A值的交易
  - 三个数据库都向协调者返回Yes
  - 数据库d_2与协调者之间的通信链路出现拥塞或故障，导致Yes消息一直没被收到
- 协调者无法收到足够的Yes，陷入无限等待，或者采用计时机制，超时后通知所有节点停止提交



总结：无法容错的根源

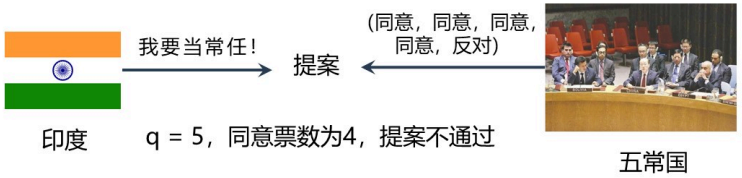
在简单的二阶段提交协议中，必须所有相关节点都进行回应后才成功提交，因此协议无法容忍任何故障



必须五常国全部同意才能通过，只要保持每次投票的同意票数为4，提案永远无法通过

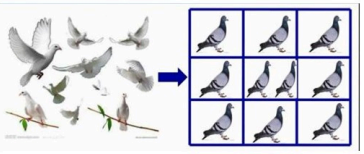
容错算法的基础：Quorum机制

- Quorum（法定人数）：某个决议被通过需要的最少人数，通常用 q 表示
- Quorum Certificate（QC）：用于证明某个决议经过某个Quorum的同意，通常为Quorum个不同的签名
- 投票（vote）：参与人通过提供自己的签名表达自己对某项提案的投票



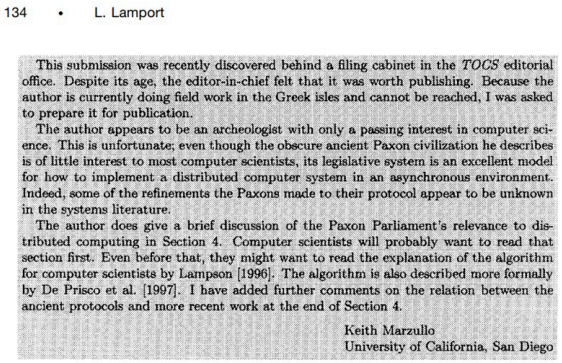
Quorum设计准则

- **鸽巢原理**：如果k+1个鸽子要放在k个鸽巢中，那么至少有一个鸽巢存在两个鸽子



- 假设N个副本，更新操作在W个副本中更新成功之后，才认为此次更新操作成功
- 对于读操作而言，至少需要读R个副本才能读到此次更新的数据。其中，W+R>N，即W和R有重叠，一般，**W+R=N+1**
- 假设系统中有5个副本，W=3，R=3。初始时数据为(V1，V1，V1，V1，V1）当某次更新操作在3个副本上成功后，认为此次更新操作成功，数据变成：(V2，V2，V2，V1，V1）最多只需要读3个副本，一定能够读到V2(此次更新成功的数据)

故障容错算法-Paxos



134 • L. Lamport

This submission was recently discovered behind a filing cabinet in the *TOCS* editorial office. Despite its age, the editor-in-chief felt that it was worth publishing. Because the author is currently doing field work in the Greek isles and cannot be reached, I was asked to prepare it for publication.

The author appears to be an archeologist with only a passing interest in computer science. This is unfortunate; even though the obscure ancient Paxon civilization he describes is of little interest to most computer scientists, its legislative system is an excellent model for how to implement a distributed computer system in an asynchronous environment. Indeed, some of the refinements the Paxons made to their protocol appear to be unknown in the systems literature.

The author does give a brief discussion of the Paxon Parliament's relevance to distributed computing in Section 4. Computer scientists will probably want to read that section first. Even before that, they might want to read the explanation of the algorithm for computer scientists by Lampson [1996]. The algorithm is also described more formally by De Prisco et al. [1997]. I have added further comments on the relation between the ancient protocols and more recent work at the end of Section 4.

Keith Marzullo
University of California, San Diego



Leslie B. Lamport
美国著名的计算机科学家，微软研究院首席研究员，著名的排版系统LaTeX的作者，拜占庭将军问题和Paxos算法的提出者，2000年获得了Dijkstra奖，2012年获得冯·诺依曼奖，2013年获得图灵奖

故障容错算法-Paxos

- 这个世界上只有一种一致性算法，那就是Paxos，其它的算法都是残次品

此（分布式）一致性非
彼（事务）一致性

——Google Chubby的作者Mike Burrows

- 问题场景：
  - 分布式数据库系统中，各节点初始状态一致
  - 部分节点可能出现故障
- 主要目的：
  - 所有正确节点按相同顺序执行相同的操作指令
  - 所有正确节点得到一致的本地状态结果
- 应用实例：
  - Cassandra, Google Spanner, Google Chubby

雨课堂 Rain Classroom

Paxos的主要思想

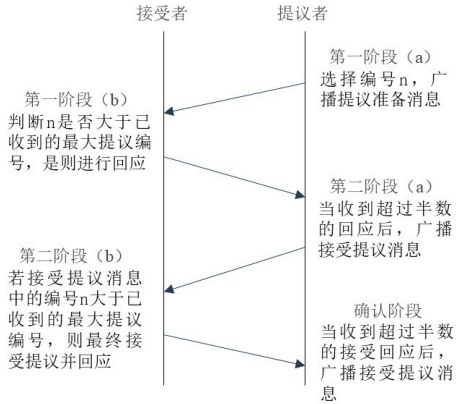
存在有Leader和无Leader两种模式，基于Quorum机制确保一致性和容错性

- Paxos算法中的3种角色，一个节点可同时担任多个角色，通常情况下由客户端担任提议者，服务端担任接受者和学习者
  - Proposer(提议者)，Acceptor(接受者)，Learner(学习者)
- Paxos的基本流程
  - 提议者为交易分配一个序号，从而形成提案，然后将其发送给所有的接受者
  - 接受者选择需要接受的提案，然后对提议者进行回应
  - 当提议者收到超过半数接受者的回应后，则通知接受者最终接受该提案
  - 接受者接受提案后告知学习者，学习者收到多数接受者信息后学习该提案

注. 详情可参考：
Lamport L. Generalized consensus and Paxos[J]. 2005.

基本的思想：无Leader模式下的运行流程

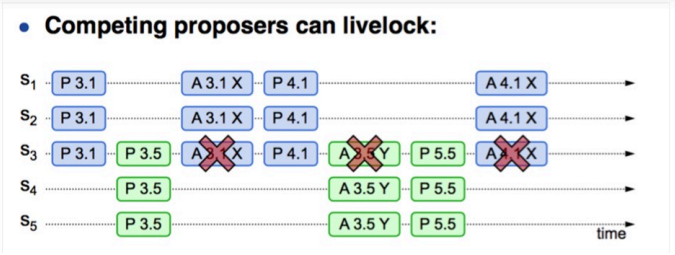
- 理想情况：无冲突出现（如右图所示），四次交互即可完成一轮共识
- 实际情况：多个提议者存在，每个提议者单独提议各自的提案；接受者面临不同提议者的提案，可能需要不断接受提议编号更大的提案
- 共识达成条件：提案完成前三次交互之前无更大提议号提案出现



一个理想的共识流程

无Leader存在的问题

每个提议者独立提议时，不仅效率低下，还可能产生活锁



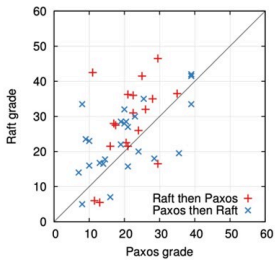
解决思路：在提议者中选取一个领导者进行提议，其它提议者不参与提议

注. Paxos及更多改进的详细介绍可参考：
Lamport L. Fast paxos[J]. Distributed Computing, 2006, 19(2): 79-103.

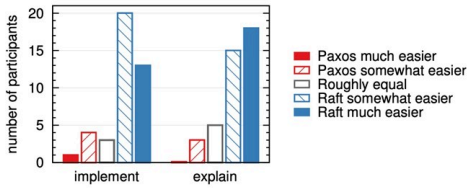
故障容错算法-Raft

目标：在保证容错的基础上，设计一个更容易理解的故障容错算法。

——Diego Ongaro and John Ousterhout



对参与学生的测试成绩



对参与学生的调查问卷

- 根据测试结果和调查问卷，说明Paxos的学习难度要远高于Raft

Raft的主要思想

Raft基于Leader完成共识过程，并基于Quorum机制确保一致性和容错性

- 在Raft中，服务器可以扮演下面角色之一：
  - Leader: 处理所有客户端交互，日志复制等，一般一次只有一个Leader
  - Follower: 类似选民，完全被动
  - Candidate: 候选人，可能被被选为新的Leader
- 一般状态下Raft运行的基本流程：
  - 所有交易被发往Leader，由Leader为其分配序号发给所有Follower
  - 当收到超过半数Follower的回应时，提交并执行交易，然后回复客户端
  - Follower提交交易后，在后续的交互中通知Follower提交相同的交易

第3节 信任问题

- **访问控制**
- 信任评价模型
- 拜占庭容错

零信任网络

“从来不信任，始终在校验” （Never Trust, Always Verify）

- 零信任模型不依靠建立隔离墙来保护可信的资源，而是接受“不可信”或“坏人”无处不在的现实，试图让全体资源都拥有自保的能力
- 零信任默认不应该信任企业网络内部和外部的任何人/设备/应用，需要基于认证和授权重构访问控制的信任基础

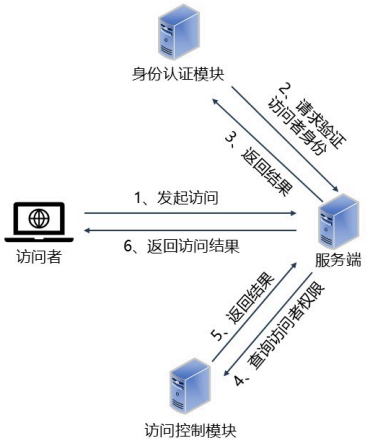


零信任对传统访问控制机制进行了范式上的颠覆
其本质是以身份为基石的动态可信访问控制

55

零信任下访问的基本流程

- 访问者发起对服务端的访问
- 服务端收到访问指令后对其进行验证，主要包含以下两个步骤
  - 请求身份认证模块验证访问者身份
  - 根据身份以及被访问资源请求访问控制模块判断访问者是否存在权限
- 若验证都通过后，服务端允许此次访问，并返回访问结果

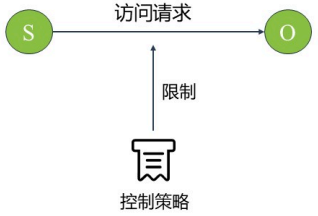


访问控制

访问控制（Access Control）指系统按用户身份，以及该身份所在的策略组来限制用户对某些信息项的访问，或限制其对某些控制功能的使用的一种技术。

- 主体S (Subject)
  - 提出对资源访问的具体请求
- 客体O (Object)
  - 提出对资源访问的具体请求
- 控制策略A (Attribution)
  - 是主体对客体的相关访问规则集合，即属性集合

- 访问控制规定了主体（用户或者进程）对客体（数据资源）的访问权限



基于角色的访问控制

RBAC设置不同的角色，并将各种身份信息与角色进行绑定，然后只对角色的访问权限进行控制，可极大地简化权限管理。

- 包含用户、角色和权限三个部分
  - 角色可以看作是一组操作的集合，不同的角色具有不同的操作集，这些操作集由系统管理员分配给角色。
  - 用户与角色是多对一或者多对多关系。一个用户对应一个角色、一个用户对应多个角色
  - 用户最终权限取多个角色权限的并集

1、根据任务需要定义具体不同的角色

2、为不同角色分配资源和操作权限

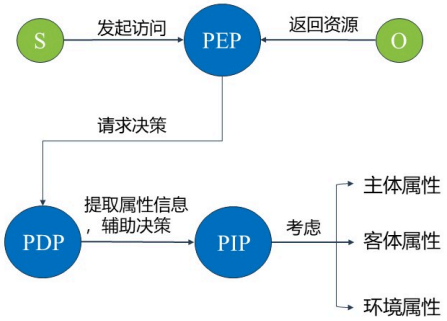
3、为一个用户组指定一个角色

RBAC模型授权管理步骤

基于属性的访问控制

用户属性指的是可以对用户进行区分的特性。用户是访问控制的主体，以其属性作为访问网络数据的依据，可以实现更细粒度的访问控制

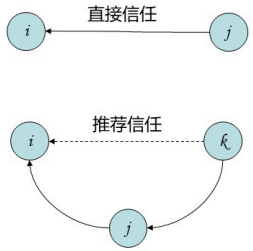
- 策略实施点 PEP：拦截访问用户的请求，并请求PDP对该请求进行评估，判断是否允许此次访问
- 策略决策点 PDP：根据访问属性及策略判断访问请求是否被允许
- 策略信息点PIP：提供属性信息，辅助访问控制



基于信任管理的访问控制

基于用户之间的初始信任和历史交互记录得到信任值，然后根据信任值等级的不同给予相应的访问权限

- 基本信任：代表着节点由于自身的属性所具有的默认信任值，依据可以是节点的角色、节点所使用的操作系统、登录地点、 地址等。
- 直接信任：由主体和客体之间的交互历史产生，直接信任随着两者的持续交互而逐渐积累。
- 推荐信任：依据信任值的传递性产生，主体可以参考其它节点与客体的交互历史对客体进行信任评价，也称作“间接信任”。



第3节 信任问题

- 访问控制
- **信任评价模型**
- 拜占庭容错

为什么需要信任评价

访问控制只能解决访问者的权限问题，但无法解决授权访问者做恶以及被访问者的信任问题

- 访问者对被访问者的访问，可能需要信任问题：
  - 文件下载
  - 资源共享
  - 访问网站链接
- 典型场景：P2P文件共享系统



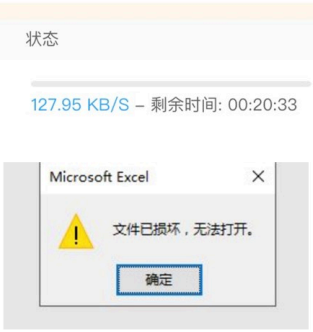
napster

gnutella

由于信任导致的P2P文件共享问题

- 资源问题导致下载速度慢
- 下载到已经损毁的垃圾文件
- 下载到附带病毒的恶意资源

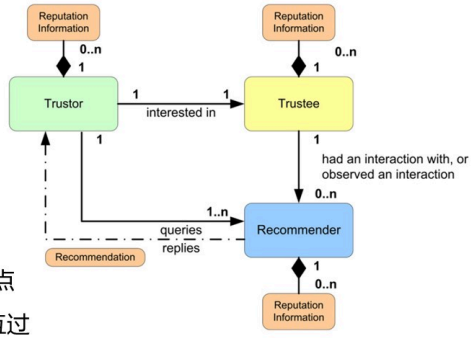




解决方式：基于信任评价模型计算节点信誉值，并根据信誉值选择资源提供者

信任评价模型的定义

- 问题：如何描述、计算、传递与更新信任
- 信任模型包含：行为模型；计算模型；传播模型
- 信任模型的角色
  - Trustee：待评价节点
  - Trustor：期待与Trustee交互节点
  - Recommender：与Trustee交互过

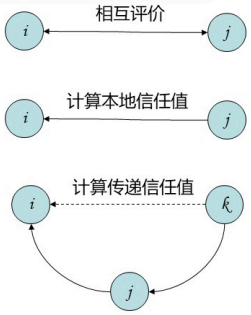


信任评价系统举例—Eigentrust

主要思想：节点对每次交互过程中的交互节点进行评价，并根据历史的交互行为评价计算节点信誉值

➢ 基本流程

- 节点根据本地的历史交互评价为每个节点计算本地信誉值
- 节点根据其它节点的本地信誉值计算出传递信誉值，并通过多次迭代确保收敛
- （可选）所有节点根据计算的传递信任值计算出全局信任值

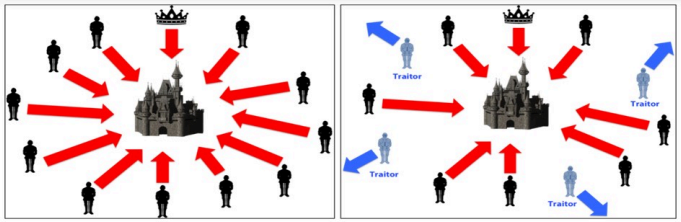


第3节 信任问题

- 访问控制
- 信任评价模型
- **拜占庭容错**

拜占庭将军问题

- **背景：**拜占庭罗马帝国疆域辽阔，许多将军守卫疆土；战时，部队相隔远，只能靠信差传递消息；至少超过半数的将军对是否共打敌人达成一致，否则失败；内部可能存在叛徒
- **问题：**存在叛徒时，忠诚的将军们如何达成共识



Coordinated Attack Leading to Victory　　Uncoordinated Attack Leading to Defeat

拜占庭将军问题定义

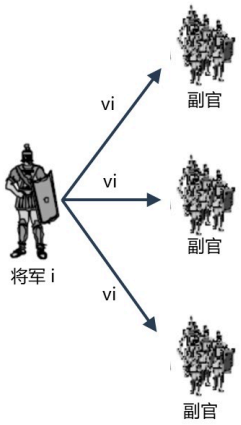
- **概念：**将军 等价于 计算机系统的部件
- **问题抽象：**
  - 每个部队由一个将军领导；
  - 一共有n个将军，其中一些是叛徒；
  - 所有军队将敌方包围，观察敌军；
  - 将军间靠通信兵交流 。
- **目标：**
  - 存在所有忠诚的将军采取同样的行动；
  - 少量叛徒无法使诚实的将军们接受一个坏的计划
- **注意：**不必识别出叛徒



Leslie B. Lamport

问题简化

- 将问题简化为：**一个将军**将消息发送给**其他的副官**
- 拜占庭将军问题
  - 将军（发布命令者）必须将他的命令发送给他的 n-1 个副官
- 交互一致性（ Interactive Consistency ， IC）
  - IC1: 所有忠诚的副官都服从同样的命令
  - IC2: 如果发布命令的将军是忠诚的，那么每个忠诚的副官都服从他的命令
- 注意：如果将军是忠诚的，那么 IC2 等价于 IC1



问题推导：一致性和正确性条件

- 一致性（等价IC1）
  - 每个将军必须得到相同的(v1, v2, ..., vn)指令向量或指令集合
  - 忠诚的将军不一定使用i将军发来的信息作为 vi，i将军有可能是叛徒

一致性条件不够充分，忠诚将军的信息有可能被修改，所以必须引入另一个正确性的条件

- 正确性（等价IC2）
  - 若i将军是忠诚的，其他忠诚的将军必须使用他发出的值作为vi

解决方法

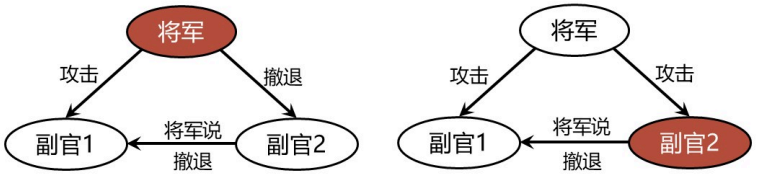
主要问题：由于将军可能是叛徒，会对不同的副官发布不同的指令，因此副官不能完全信任将军的指令

解决思路：副官在收到将军的指令后，互相交换从将军处收到的指令值，再基于交互结果选择最终执行指令

- 两种方法
  - 口头消息协议：所有副官在交换从将军处收到的指令值时基于口头传播，因此可以说谎（对应于系统中**无**数字签名的场景）
  - 签名消息协议：副官从将军处收到的指令附带签名，因此无法伪造将军的指令，将军也无法对已发布的指令抵赖（对应于系统中**有**数字签名的场景）

口头消息下三将军问题是无解的

- 现有三个将军，其中一个是叛徒
  - 两种消息：攻击、撤退
  - 深色的是叛徒
- 副官1在两种情形下收到的消息是一样的，无法判断谁是叛徒
  - 左图中：根据IC1，副官1必须攻击
  - 右图中：副官1根据IC1判断必须攻击，但将军和副官2都撤退了，于是副官1被出卖了



口头消息(OM)有解的必要条件

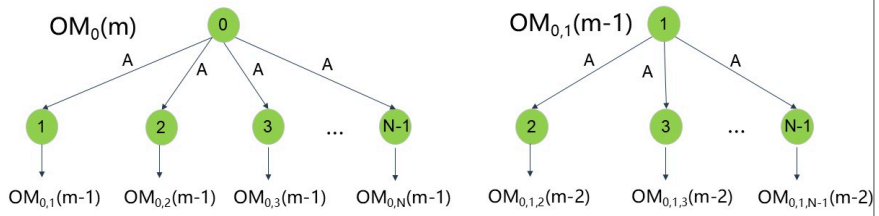
结论：假设将军个数为N，且最多存在m个叛徒时，当N≥3m+1时，OM(m)算法可以确保所有忠诚的将军达成一致

- 假设
  - 每个消息都被正确发送
  - 被发送的消息在回合结束前都能被正确接收（同步假设）
  - 消息的接收者知道谁是发送者
- 确保
  - 叛徒不能通过说谎来影响共识
  - 叛徒不能通过保持沉默来影响共识

证明思想：根据数学归纳法完成证明，即m=1时成立，并且当m=k时成立，能推导出m=k+1时也成立

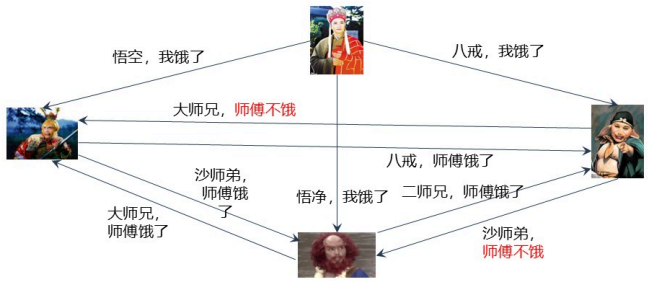
口头消息协议

- 假设N个副官编号分别为0,1,...,N-1，初始状态下0为将军，并执行$OM_0(m)$
- $OM_0(m)$的运行流程为：将军0给其它副官发送指令，然后其它副官分别执行$OM_{0,i}(m-1)$，将从将军处收到的指令发给剩余副官
- $OM_{k1,k2,...,kn,i}(m)$表示的OM算法是在除去副官k1,k2,...,kn后剩余的副官中执行，且副官i将从副官kn执行$OM_{k1,k2,...,kn}(m+1)$时发送的指令发给剩余副官



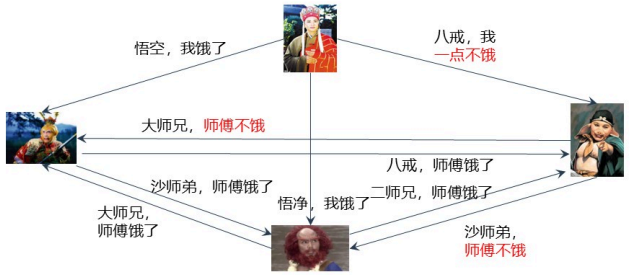
四将军问题实例：懒惰的八戒

- 悟空和沙僧都收到了（饿了，饿了，不饿），最后都认定师傅饿了
- 满足了IC1，师傅、悟空、沙僧的意见是一致的
- 满足了IC2，悟空和沙僧都听师傅的



四将军问题实例：机智的师傅

- 悟空、沙僧、八戒都收到了（饿了，饿了，不饿），都认定师傅饿了
- 满足了IC1，悟空、沙僧、八戒的意见是一致的
- 无需遵守师傅的指令(即无需满足了IC2)，师傅撒谎了



口头消息的缺点

- 开销巨大
  - OM(m) 调用 n – 1 次 OM(m-1)
  - OM(m-1) 调用 n – 2 次OM(m-2)
  - OM(m-2) 调用 n – 3 次OM(m-3)
  - ……
  - OM(m-k) 被调用 n-1, n-2, …, n-k次
  - $O(n^m)$
- 口头消息不能溯源（无法确定消息是否被篡改），导致需要传输大量的信息

方案二：签名消息(SM)协议

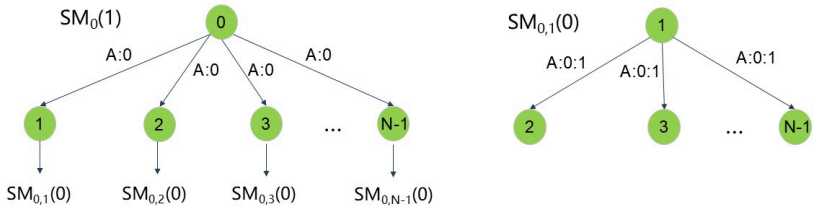
改进：在实际系统中，可以引入签名机制确保节点发送的消息无法被篡改，从而得到签名消息算法SM(Signed Messages)

- 在口头消息协议的基础上添加假设
  - 忠诚将军的签名不可伪造
  - 任何人都可以验证将军的签名
- 确保
  - 叛徒不能通过说谎来影响共识
  - 叛徒不能通过保持沉默来影响共识

结论：假设将军个数为N，且最多存在m个叛徒时，当N≥2m+1时，SM(1)算法可以确保所有忠诚的将军达成一致

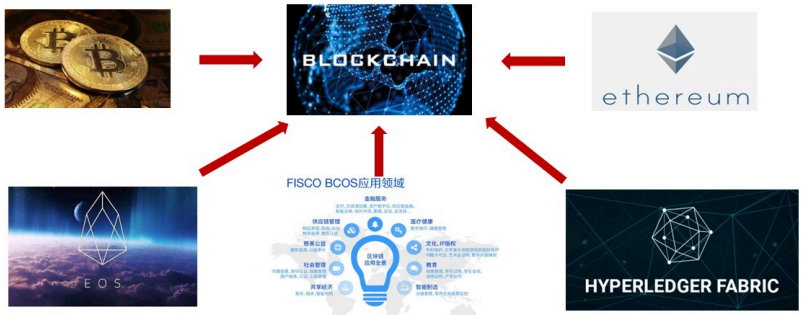
签名消息(SM)协议

- 假设N个副官编号分别为0,1,…,N-1，初始状态下0为将军，并执行$SM_0(1)$，从而给其它副官发送附带签名的指令x:0
- 收到x:0后，副官i执行$SM_{0,i}(0)$，即在签名指令x:0上附带自己签名得到x:0:i，然后将其发送给其它副官
- 每个副官收集来自所有副官的指令（包括自己和将军在内），当出现不同指令时则撤退（说明将军是叛徒）；否则当超过半数为相同指令时选择该指令



拜占庭容错共识的典型应用场景: 区块链

区块链：构造一个对客户端来说可信的分布式账本；容忍少数拜占庭节点作恶解决分布式系统不同节点间协作时的信任问题



第4节 总结

主要内容

剖析分布式系统安全问题的根源，探讨如何从交互网络、分布式算法和信任三方面构建安全、稳定、可信的分布式系统



总结

- 我们首先回顾了分布式系统的起源与发展到如今的无处不在；而后，我们将分布式系统的组成氛围了三大组成部分：**交互网络，分布式算法和信任机制；**分布式系统的安全根源也体现在分布式系统的三个组成部分中

- 对于交互网络安全，我们介绍了交互信道隐患及解决措施、网络故障与弹性Overlay网络、针对Overlay网络的日蚀攻击与防御机制

- 对于分布式算法安全，我们介绍了时钟同步、并发、故障容错三方面出发，介绍了Cristian、Berkely、NTP等常见的时钟同步算法，介绍了两阶段提交、锁机制等并发控制手段，介绍了Paxos、Raft等故障容错的一致性算法

- 对于信任问题，我们介绍了访问控制、信任评价模型、拜占庭容错等常见的解决信任的方法

雨课堂 Rain Classroom